HAI913I
Evolution et restructuration des logiciels

TP2 - Compréhension et architecture

Nathan Boureux - M2 GL

N° étudiant : 22206444

Enseignant responsable : Bachar Rima

Année universitaire 2026–2027



Table des matières

1 Objectif et périmètre **1**

2 Conception générale **1**

3 Conventions appliquées **2**

4 Réponses **2**

4.1 Partie A 2

4.1.1 A1 2

4.1.2 A2 2

4.2 Partie B 3

4.2.1 B1 3

4.2.2 B2 3

4.2.3 B3 3

5 Architecture et diagrammes **4**

6 Validation **6**

6.1 Étape A0 6

6.2 Étape A1 7

6.3 Étape A2 8

6.4 Étape B1 9

6.5 Étape B2 9

6.6 Étape B3 10

7 Interprétation **11**

7.1 Question 1 11

7.2 Question 2 11

7.3 Question 3 11

7.4 Question 4 12

7.5 Question 5 12

8 Difficultés **12**

9 Utilisation de l'IA **13**

10 Conclusion **13**

11 Source **13**

1 Objectif et périmètre

L'outil développé au cours de ce tp charge les faits (classes, méthodes, appels) extraits grâce au projet du tp précédent, calcule le couplage entre les classes, construit un arbre de clustering hiérarchique ascendant, et déduit des modules architecturaux en coupant cet arbre selon un seuil paramétrable.

Il ne parse pas de code source Java directement (c'est le rôle du tp1) et ne prend pas en compte l'héritage, le typage ou l'instanciation dans le calcul du couplage.

2 Conception générale

Ce projet est basé sur un pipeline représentant le projet où chaque étape est une classe, exécutées les unes à la suite des autres. Tout d'abord, la classe `FactsCsvExporter` ajoutée au tp1 exporte les données nécessaires sous forme de fichiers csv, directement dans le dossier `resources_TP1` de ce projet.

— **Étape A1 :** La classe `CouplingAnalysis` est chargée du filtrage des arcs intra-classes et du calcul de la métrique de couplage $C(A, B)$ pour chaque paire de classes distinctes.

— **Étape A2 :** La classe `WeightedCouplingGraph` transforme les résultats du couplage en un graphe pondéré. Elle gère également l'export en csv et dot.

— **Étape B1 :** La classe `HierarchicalClustering` implémente le regroupement hiérarchique ascendant. Elle calcule dynamiquement les similarités entre groupes (critère du lien moyen) et applique strictement la règle de départage déterministe lors des égalités.

— **Étape B2 :** La classe `Dendrogram` modélise l'historique des fusions sous la forme d'un arbre binaire en mémoire, composé d'instances de `DendrogramNode` (feuilles et nœuds internes). Elle exporte ensuite cet arbre sous forme texte (indenté) et dot.

— **Étape B3 :** Enfin, la classe `ModuleIdentifier` parcourt cet arbre récursivement pour identifier les modules candidats en fonction du seuil de couplage `CP` paramétré par l'utilisateur, et vérifie la contrainte $M/2$.

3 Conventions appliquées

Seuls les appels de méthodes sont comptés comme relation entre deux classes.

Couplage : $C(A,B) = \dfrac{r(A,B)}{|E_{inter}|}$, où E_{inter} exclut les arcs internes.

Le critère de liaison est le lien moyen, c'est à dire la somme de tous les couplages divisée par $|G_1| \times |G_2|$.

Pour le départage : ID croissant (0 à M-1 ordre alphabétique), puis choix du couple au plus petit ID lexicographique.

Coupe : le couplage interne moyen est strictement supérieur au seuil CP : $\overline{C}(G) > CP$. Il est calculé par la formule $\overline{C}(G) = \dfrac{1}{\binom{|G|}{2}} \sum_{\{A,B\} \subseteq G, A \neq B} C(A,B)$

4 Réponses

4.1 Partie A

4.1.1 A1

La classe `CouplingAnalysis` parcoure les appels pour compter $|E_{inter}|$ en excluant les appels internes. Les arcs restants sont agrégés dans un couple de classes non orienté (`ClassPair`), garantissant la symétrie.

On détermine ainsi le nombre d'arcs $r(A,B)$ et on calcule le couplage via la formule $C(A,B) = \dfrac{r(A,B)}{|E_{inter}|}$. (Voir conventions 3)

4.1.2 A2

À la place d'une matrice dense remplie de zéros, le graphe est stocké en mémoire sous forme de liste de nœuds et d'arêtes. Lors de la construction, le degré de chaque sommet est vérifié pour identifier facilement les classes isolées. La classe gère ensuite les exports : un fichier csv et un fichier dot.

4.2 Partie B

4.2.1 B1

L'implémentation repose sur une liste des groupes actifs, mise à jour à chaque itération. À chaque étape, une double boucle évalue la similarité entre toutes les paires de groupes actifs. Cette similarité est calculée selon le critère du lien moyen, en divisant la somme totale des couplages inter-groupes par le produit des tailles des deux groupes.

Les deux groupes les plus similaires sont fusionnés, retirés des groupes actifs, la fusion est enregistrée et le groupe formé par celle-ci est ajouté à la liste des groupes actifs.

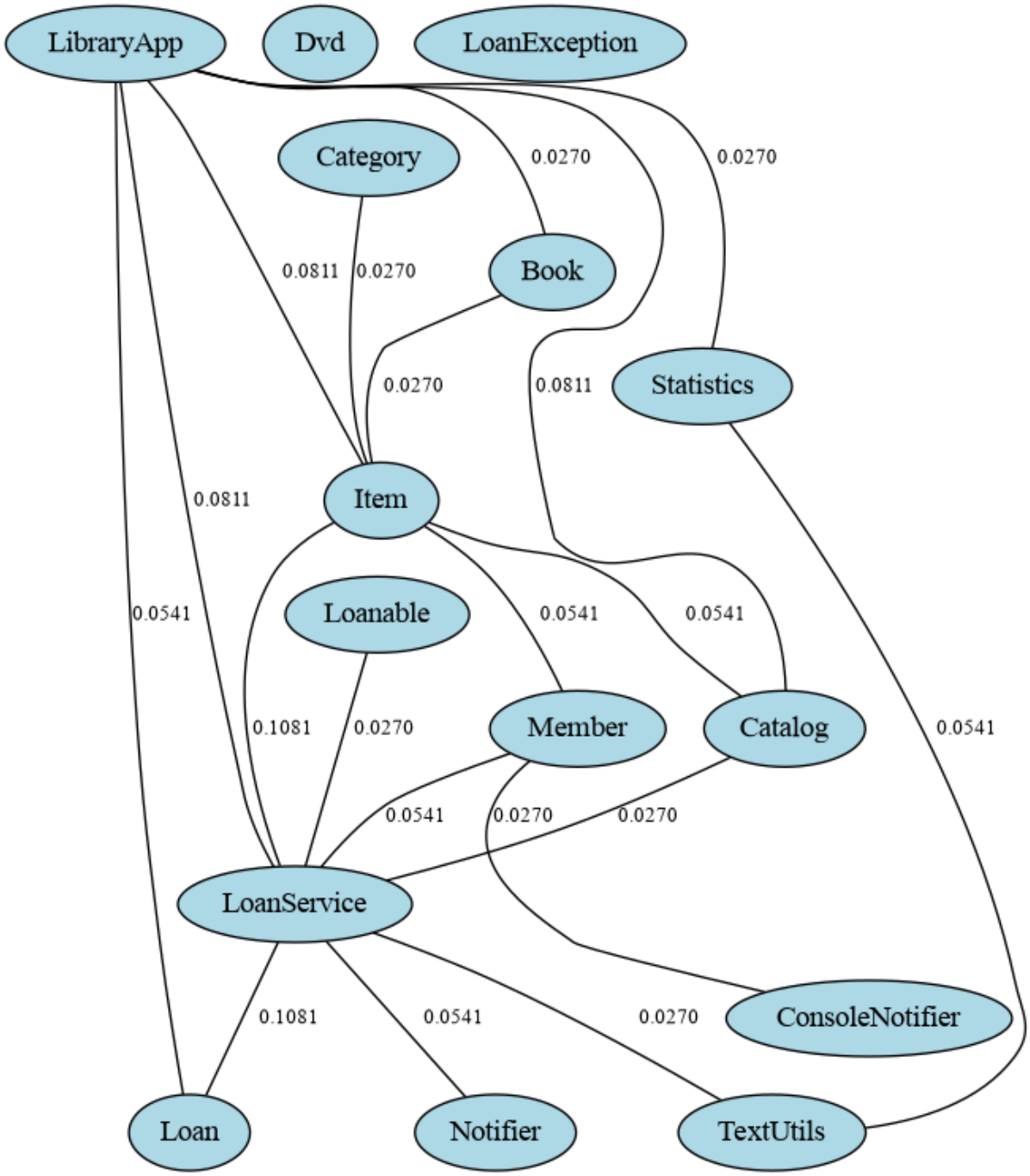
4.2.2 B2

Le dendrogramme généré est un arbre binaire classique où les feuilles représentent les 15 classes originelles et les nœuds représentent les fusions, portant chacun la valeur de similarité à laquelle le regroupement a eu lieu. La racine de cet arbre rassemble toutes les classes. Voir partie 5

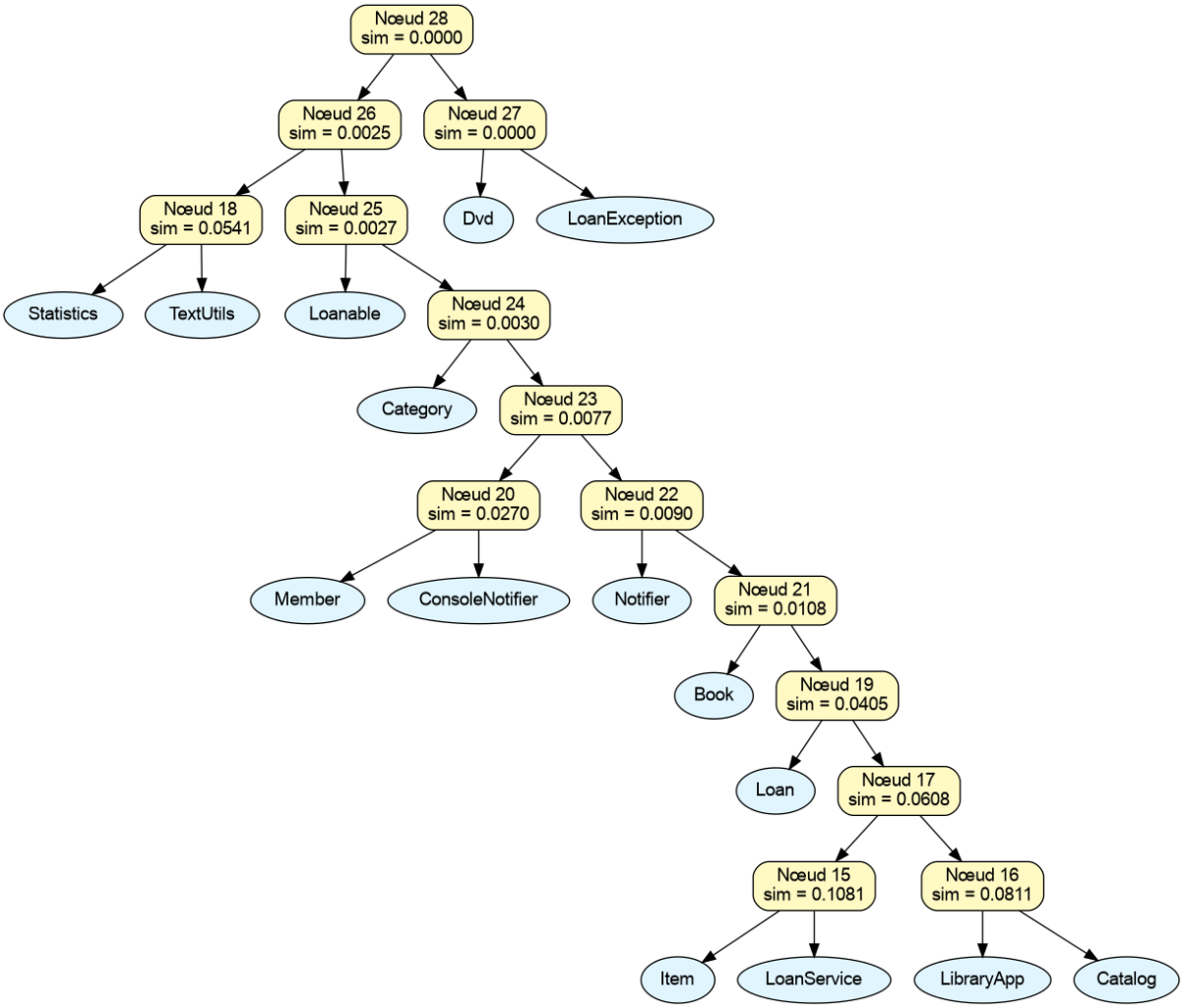
4.2.3 B3

La coupe s'effectue via un parcours récursif de l'arbre en partant de la racine. Pour chaque nœud visité, le couplage interne moyen $\overline{C}(G)$ est calculé dynamiquement sur toutes ses classes membres (Voir conventions 3). Si cette valeur dépasse strictement le paramètre utilisateur CP, le sous-arbre entier est retenu comme module. Dans le cas contraire, le parcours descend vers les enfants.

5 Architecture et diagrammes



Graphe de couplage pondéré



Dendrogramme

6 Validation

6.1 Étape A0

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running hai913i.tp2.FactsTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.050 s -- in hai913i.tp2.FactsTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
```

Le mvn clean test passe

```
nathan@Discov3rWOrld:~/S9/ERL/TP_comprehension_architecture/starter$ java -jar target/hai913i-tp2.jar ~/S9/ERL/TP_analyse_statique/starter/output-facts/
Dossier des faits   : /home/nathan/S9/ERL/TP_analyse_statique/starter/output-facts
Jeu de données      : facts-projet
Classes             : 15
Méthodes            : 59
Arcs du graphe      : 44
```

Le programme annonce bien 15 classes, 59 méthodes et 44 arcs sur le jeu de données du projet de validation (données extraites par le tp1)

```
nathan@Discov3rWOrld:~/S9/ERL/TP_comprehension_architecture/starter$ java -jar target/hai913i-tp2.jar ../examples/
Dossier des faits   : /home/nathan/Fac/2026/Cours/S9/ERL/TP_comprehension_architecture/starter/../examples
Jeu de données      : facts-mini
Classes             : 3
Méthodes            : 5
Arcs du graphe      : 4
```

Il annonce bien 3 classes, 5 méthodes, 4 arcs sur le micro jeu de données

6.2 Étape A1

```
nathan@Discov3rW0rld:~/S9/ERL/TP_comprehension_architecture/starter$ java -jar target/hai913i-tp2.jar ~/S9/ERL/TP_analyse_statique/starter/output-facts/
Dossier des faits   : /home/nathan/S9/ERL/TP_analyse_statique/starter/output-facts
Jeu de données      : facts-projet
Classes             : 15
Méthodes            : 59
Arcs du graphe      : 44
Seuil CP retenu     : 0.0250

Faits chargés. Les étapes A2 à B3 restent à écrire.

--- RÉSULTATS ÉTAPE A1 ---
|E_inter|                 : 37
Arcs écartés              : 7
Somme des couplages       : 1.0000
Couples non nuls          : 19
Total couples possibles   : 105

--- COUPLES PAR COUPLAGE DÉCROISSANT ---

  - library.model.Item <---> library.service.LoanService : r = 4, C = 0.1081
  - library.service.LoanService <---> library.service.LoanService.Loan : r = 4, C = 0.1081
  - library.app.LibraryApp <---> library.service.Catalog : r = 3, C = 0.0811
  - library.app.LibraryApp <---> library.model.Item : r = 3, C = 0.0811
  - library.app.LibraryApp <---> library.service.LoanService : r = 3, C = 0.0811
  - library.app.LibraryApp <---> library.service.LoanService.Loan : r = 2, C = 0.0541
  - library.model.Item <---> library.model.Member : r = 2, C = 0.0541
  - library.model.Item <---> library.service.Catalog : r = 2, C = 0.0541
  - library.model.Member <---> library.service.LoanService : r = 2, C = 0.0541
  - library.service.LoanService <---> library.service.Notifier : r = 2, C = 0.0541
  - library.util.Statistics <---> library.util.TextUtils : r = 2, C = 0.0541
  - library.app.LibraryApp <---> library.model.Book : r = 1, C = 0.0270
  - library.app.LibraryApp <---> library.util.Statistics : r = 1, C = 0.0270
  - library.model.Book <---> library.model.Item : r = 1, C = 0.0270
  - library.model.Category <---> library.model.Item : r = 1, C = 0.0270
  - library.model.Member <---> library.service.ConsoleNotifier : r = 1, C = 0.0270
  - library.service.Catalog <---> library.service.LoanService : r = 1, C = 0.0270
  - library.model.Loanable <---> library.service.LoanService : r = 1, C = 0.0270
  - library.service.LoanService <---> library.util.TextUtils : r = 1, C = 0.0270
```

On peut voir sur les résultats : 7 arcs écartés (internes), |Einter| = 37, 19 couples non nuls dont la somme vaut 1,000, 105 couples possibles.
Le couple le plus couplé est Item – LoanService avec r = 4, et C = 0,1081.
Dvd et LoanException n'apparaissent dans aucun couple.

6.3 Étape A2

```
--- RÉSULTATS ÉTAPE A2 : GRAPHE DE COUPLAGE ---
Sommets    : 15
Arêtes     : 19

Les 5 arêtes les plus lourdes :
  1. library.model.Item <---> library.service.LoanService (r = 4, weight = 0.1081)
  2. library.service.LoanService <---> library.service.LoanService.Loan (r = 4, weight = 0.1081)
  3. library.app.LibraryApp <---> library.service.Catalog (r = 3, weight = 0.0811)
  4. library.app.LibraryApp <---> library.model.Item (r = 3, weight = 0.0811)
  5. library.app.LibraryApp <---> library.service.LoanService (r = 3, weight = 0.0811)

Classes isolées : 2
  - library.model.Dvd
  - library.service.LoanException

Fichier csv exporté dans : outputs/coupling-graph.csv

Fichier dot exporté dans : outputs/coupling-graph.dot

Rechargement du CSV : 19 arêtes, 15 sommets
Les graphes ne sont pas les mêmes : graph == reloadedGraph : false
Sommaire du reloadedGraph :

--- RÉSULTATS ÉTAPE A2 : GRAPHE DE COUPLAGE ---
Sommets    : 15
Arêtes     : 19

Les 5 arêtes les plus lourdes :
  1. library.model.Item <---> library.service.LoanService (r = 4, weight = 0.1081)
  2. library.service.LoanService <---> library.service.LoanService.Loan (r = 4, weight = 0.1081)
  3. library.app.LibraryApp <---> library.service.Catalog (r = 3, weight = 0.0811)
  4. library.app.LibraryApp <---> library.model.Item (r = 3, weight = 0.0811)
  5. library.app.LibraryApp <---> library.service.LoanService (r = 3, weight = 0.0811)

Classes isolées : 2
  - library.model.Dvd
  - library.service.LoanException
nathan@Discov3rW0rld:~/S9/ERL/TP_comprehension_architecture/starter$
```

On peut voir sur les résultats : 15 sommets et 19 arêtes. Les cinq arêtes les plus lourdes sont Item–LoanService, LoanService–Loan, LibraryApp–Item, LibraryApp–Catalog et LibraryApp–LoanService. Deux nœuds sont isolés : Dvd et LoanException
Le graphe est bien exporté dans deux formats, puis rechargé et l'affichage du graphe rechargé est le même.

6.4 Étape B1

```
--- RÉSULTATS ÉTAPE B1 : CLUSTERING ASCENDANT ---
Nombre de fusions effectuées : 14

Étape | ID A   | ID B   | Similarité | Groupe A                                                
```

6.6 Étape B3

```
--- RÉSULTATS ÉTAPE B3 : IDENTIFICATION DES MODULES ---
Seuil CP retenu    : 0.0150
Nombre de modules  : 4
M / 2              : 7
Contrainte M/2     : RESPECTÉE

--- DÉTAIL DES MODULES CANDIDATS ---
Module   | Nœud ID  | Nb Classes   | Couplage Interne   | Classes membres
-------------------------------------------------------------------------------------------------------------------
Mod 1    | Nœud 18  | 2            | 0.0541             | {Statistics, TextUtils}
Mod 2    | Nœud 25  | 11           | 0.0162             | {LibraryApp, Book, Category, Item, Loanable, Member, Catalog, ConsoleNotifier, LoanService, Loan, Notifier}
Mod 3    | Nœud 3   | 1            | 0.0000             | {Dvd}
Mod 4    | Nœud 9   | 1            | 0.0000             | {LoanException}
nathan@Discov3rW0rld:~/S9/ERL/TP_comprehension_architecture/starter$
```

CP = 0,015, la contrainte est respectée, 4 modules mais on a le module 2 de 11 classes

```
--- RÉSULTATS ÉTAPE B3 : IDENTIFICATION DES MODULES ---
Seuil CP retenu    : 0.0250
Nombre de modules  : 7
M / 2              : 7
Contrainte M/2     : RESPECTÉE

--- DÉTAIL DES MODULES CANDIDATS ---
Module   | Nœud ID  | Nb Classes   | Couplage Interne   | Classes membres
-------------------------------------------------------------------------------------------------------------------
Mod 1    | Nœud 18  | 2            | 0.0541             | {Statistics, TextUtils}
Mod 2    | Nœud 5   | 1            | 0.0000             | {Loanable}
Mod 3    | Nœud 2   | 1            | 0.0000             | {Category}
Mod 4    | Nœud 20  | 2            | 0.0270             | {Member, ConsoleNotifier}
Mod 5    | Nœud 22  | 7            | 0.0335             | {LibraryApp, Book, Item, Catalog, LoanService, Loan, Notifier}
Mod 6    | Nœud 3   | 1            | 0.0000             | {Dvd}
Mod 7    | Nœud 9   | 1            | 0.0000             | {LoanException}
nathan@Discov3rW0rld:~/S9/ERL/TP_comprehension_architecture/starter$
```

CP = 0,025, la contrainte est respectée, 7 modules

```
--- RÉSULTATS ÉTAPE B3 : IDENTIFICATION DES MODULES ---
Seuil CP retenu    : 0.0450
Nombre de modules  : 10
M / 2              : 7
Contrainte M/2     : VIOLÉE, cette coupe est rejetée

--- DÉTAIL DES MODULES CANDIDATS ---
Module   | Nœud ID  | Nb Classes   | Couplage Interne   | Classes membres
-------------------------------------------------------------------------------------------------------------------
Mod 1    | Nœud 18  | 2            | 0.0541             | {Statistics, TextUtils}
Mod 2    | Nœud 5   | 1            | 0.0000             | {Loanable}
Mod 3    | Nœud 2   | 1            | 0.0000             | {Category}
Mod 4    | Nœud 6   | 1            | 0.0000             | {Member}
Mod 5    | Nœud 8   | 1            | 0.0000             | {ConsoleNotifier}
Mod 6    | Nœud 12  | 1            | 0.0000             | {Notifier}
Mod 7    | Nœud 1   | 1            | 0.0000             | {Book}
Mod 8    | Nœud 19  | 5            | 0.0595             | {LibraryApp, Item, Catalog, LoanService, Loan}
Mod 9    | Nœud 3   | 1            | 0.0000             | {Dvd}
Mod 10   | Nœud 9   | 1            | 0.0000             | {LoanException}
nathan@Discov3rW0rld:~/S9/ERL/TP_comprehension_architecture/starter$
```

CP = 0,045, la contrainte est violée (comme on peut le voir), 10 modules

7 Interprétation

7.1 Question 1

Le module 1 `{Statistics, TextUtils}` correspond au paquetage `library.util`. Les classes utilitaires travaillent ensemble et n'ont pas de dépendance vers le reste de l'application.

Mais le reste diverge fortement : Le gros module 5 regroupe `{LibraryApp, Book, Item, Catalog, LoanService, Loan, Notifier}`. Il mélange la couche app (interface), la couche model (données) et la couche service (logique métier).

Cette divergence est due au fait que la mesure de couplage groupe les classes selon le flux d'exécution : `LibraryApp` appelle `Catalog` et `LoanService` qui manipulent `Item` et `Book`, contrairement aux paquetages Java, qui séparent les responsabilités techniques (Vue, Modèle, Contrôleur). Ce sont deux préoccupations différentes, l'analyse statique des appels regroupe le code par "cas d'usage" (use-case) plutôt que par couche technique.

7.2 Question 2

Définition du § 5.1 : Deux classes sont en relation lorsqu'une méthode de l'une appelle une méthode de l'autre. L'instanciation (new), l'héritage, le typage et la levée d'exceptions ne comptent pas.

Ici, `LoanException` est instanciée et lancée, mais aucune de ses méthodes n'est appelée. `Dvd` est instancié et manipulé seulement à travers l'interface `Item`, mais aucune méthode spécifique à `Dvd` n'est appelée directement par une autre classe. La relation de couplage stricte du TP les rend donc invisibles.

7.3 Question 3

Le module 4 `{Member, ConsoleNotifier}` peut surprendre. Conceptuellement, un membre (model) n'a rien à voir avec le fait de notifier via la console (service).

L'explication algorithmique : En regardant les sorties, on voit que le couplage `Member <--> ConsoleNotifier` vaut 0.0270. `ConsoleNotifier` possède une méthode `send(Member m, ...)` qui appelle `m.getName()`. Comme ces deux classes n'ont que peu d'autres liens forts pour les retenir ailleurs au début de l'algorithme, elles ont été regroupées à l'étape 6 avec une similarité de 0.0270.

7.4 Question 4

Résultats :

— $CP = 0.015 \rightarrow 4$ modules. L'exigence est trop faible, ce qui produit un module géant de 11 classes.

— $CP = 0.045 \rightarrow 10$ modules. L'exigence est trop forte, l'application est fragmentée en singletons isolés et violant la contrainte $M/2$ ($10 > 7$).

Troisième résultat et valeur retenue : $CP = 0.025$. C'est la valeur optimale car elle satisfait la contrainte ($7 \leq M/2$), avec un équilibre entre un "noyau" (les classes liées aux prêts, qui forment un gros groupe) et l'isolement correct des classes utilitaires et de la gestion des membres.

7.5 Question 5

Limites de la relation comptée : En ignorant l'héritage et le polymorphisme, des composants fortement liés (une interface et son implémentation) sont ignorés. Une classe appelée uniquement via son interface parente paraîtra isolée (ex : `Dvd`).

Limites du critère de liaison (lien moyen) : Le lien moyen est influencé par la taille des groupes. Dès qu'un groupe devient gros, le dénominateur ($|G_1| \times |G_2|$) augmente fortement. Cela empêche l'addition de nouvelles classes car la similarité diminue très rapidement vers 0, même si une classe a un appel très fort vers une classe spécifique de ce groupe.

Limites du seuil CP : CP est une valeur globale et statique, donnant le même niveau d'exigence à tout l'arbre, ce qui ne peut pas fonctionner : un petit groupe "utilitaire" très cohésif nécessiterait un CP élevé, tandis que la couche métier centrale tolérerait un CP plus bas. Il faudrait des seuils différents pour des parties différentes de l'arbre.

8 Difficultés

J'ai rencontré quelques difficultés, notamment au début pour l'étape A0, pour que le programme me donne 59 méthodes et 44 arcs. Il renvoyait trop de méthodes et 1 arc de trop, mais pour le cas des méthodes la cause était une erreur dans l'exportateur du tp1. Pour le cas des arcs, un arc existait pour la classe LoanException. Cependant il ne soit pas être compté car il n'y a pas d'appel de méthode, seulement une instanciation.

Une autre difficulté qui est en réalité plutôt une subtilité, lors du clustering hiérarchique, on calcule la similarité maximale entre deux groupes. Les similarités sont des nombres flottants (calculés via des divisions), il peut y avoir des legères erreurs d'arrondi. Il existe donc une contrainte $\epsilon = 10^{-9}$. Si la différence de similarités est inférieure, les similarités sont considérées comme égales.

9 Utilisation de l'IA

Gemini 3.1 Pro a été utilisé ponctuellement comme assistant de développement, principalement pour du débuggage et pour l'explication des formules mathématiques impliquées dans ce tp (comprendre à quoi chaque membre d'une formule correspondait). L'utilisation de records comme structure de stockage a aussi été suggérée par l'IA.

10 Conclusion

Quelques améliorations possibles seraient dans un premier temps la prise en compte de l'héritage et du typage des attributs pour enrichir le graphe d'appel (comme proposé dans le Bonus C2), et dans un second temps déterminer le seuil CP dynamiquement en fonction de la profondeur de l'arbre.

11 Source

Dépot : https://github.com/NathanBrx/TP_comprehension_architecture